Automated Java Application Deployment Using Jenkins CI/CD

❖ Project Overview

This project demonstrates a **multi-node CI/CD automation workflow using Jenkins** to automatically build and deploy a Java web application onto an Apache **Tomcat Server** hosted on a separate AWS EC2 instance.

The system follows a complete DevOps workflow:

1. Source Code Management (SCM): Java application code is stored in GitHub.
2. Continuous Integration (CI): Jenkins pulls the code, compiles it using Maven (or other build tools), and generates a WAR file.
3. Artifact Delivery: WAR file is transferred from the Jenkins build node to the Tomcat application server.
4. Continuous Deployment (CD): Tomcat automatically deploys the WAR file, making the application available to users.

The setup uses **two separate AWS EC2 Ubuntu servers** to simulate a real multi-node production environment:

- **Jenkins Server (Master Node)** – Handles build and deployment automation.
- **Tomcat Server (Worker/Deployment Node)** – Hosts the deployed web application.

This project covers key DevOps concepts:
automation, artifact packaging, remote deployment, Jenkins multi-node configuration, and application hosting on Tomcat.

❖ Objective of the Project

To build an automated CI/CD workflow that:

- Fetches Java code from GitHub
- Builds a .war artifact
- Deploys it to a remote Tomcat server
- Uses Jenkins master → worker node communication
- Ensures a reliable automated delivery workflow

❖ Why Multi-Node Architecture?

Using two different servers provides:

1. Real DevOps Production-Like Setup

Different teams (CI team & Ops team) handle separate servers.

2. Better Security

Jenkins does not run application servers directly; builds and deployments happen on dedicated nodes.

3. Scalability

More Tomcat nodes can be added later as Jenkins workers.

4. Reliable Deployment Flow

Even if Jenkins restarts, Tomcat continues serving the application.

- **Prerequisites**

1. AWS Infrastructure Setup

Create two Ubuntu EC2 instances**:**

a) Jenkins Server

- Ubuntu 20.04/22.04
- Installed Jenkins

- Configured Java, Git, Maven

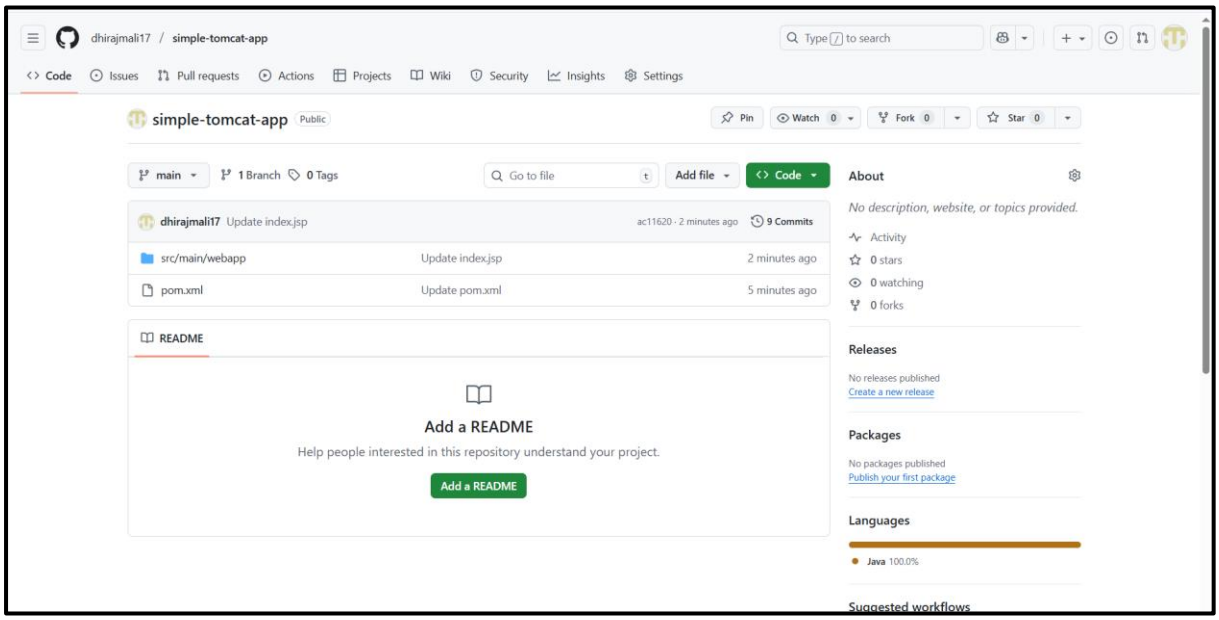
b) Tomcat Server

- Ubuntu 20.04/22.04
- Installed Apache Tomcat (v9/v10)
- Enabled /manager application
- Created tomcat user with deployment permissions
- Configured firewall rules / security groups

2. GitHub Repo Setup

repository contains:

- Java source code
- pom.xml (for Maven build)



Step 1: Setup Tomcat Hosted Instance as Jenkins Worker Node

1. Configure Passwordless SSH Between Jenkins Master and Tomcat Worker Node

To add the Tomcat EC2 instance as a Jenkins worker node, **passwordless SSH authentication** must be configured. This allows Jenkins Master to communicate securely with the worker without requiring a manual password each time.

In this step, we generate an SSH key on the Jenkins server and copy the public key to the Tomcat server user (demo), enabling secure, automated access.

a) Generate SSH Key Pair on Jenkins Master

Login to your **Jenkins server** (master node) and run: **ssh-keygen**

This creates:

- Private key → /var/lib/jenkins/.ssh/id_ed25519
- Public key → /var/lib/jenkins/.ssh/id_ed25519.pub

This key will be used by Jenkins to access the Tomcat server without entering a password.

```
jenkins@ip-172-31-21-178:~$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/var/lib/jenkins/.ssh/id_ed25519):
Created directory '/var/lib/jenkins/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /var/lib/jenkins/.ssh/id_ed25519
Your public key has been saved in /var/lib/jenkins/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:gO6pzorxSrZvphqQCQQnOKBOuh6uyN2II3PX1VDjAws jenkins@ip-172-31-21-178
The key's randomart image is:
+--[ED25519 256]--+
|B..              |
|=o   .E . o      |
|oo  . .. = .     |
|=o .   .o o      |
|=.  .   So .     |
|.. . .  . .      |
|++  o. .         |
|@**++ .          |
|@&@= .           |
+----[SHA256]-----+
jenkins@ip-172-31-21-178:~$
```

b) Copy the Public Key to the Tomcat Server

Use the following command to copy the public key to the **Tomcat instance**:

ssh-copy-id demo@<Tomcat-Server-IP>

- Enter the password of the **demo** user once.
- This adds the key to:

/home/demo/.ssh/authorized_keys

This confirms the key was successfully installed.

```
jenkins@ip-172-31-21-178:~$ ssh-copy-id demo@34.204.49.77
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/var/lib/jenkins/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
demo@34.204.49.77's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'demo@34.204.49.77'"
and check to make sure that only the key(s) you wanted were added.

jenkins@ip-172-31-21-178:~$
```

c) Verify Passwordless SSH Login

Now test if the Jenkins master can SSH into the Tomcat server **without password**:

ssh demo@<Tomcat-Server-IP>

Example: **ssh demo@34.204.49.77**

Successfully logged in, as shown in the screenshot.

This confirms **passwordless SSH access is working**.

Finally exit: **exit**

```
jenkins@ip-172-31-21-178:~$ ssh demo@34.204.49.77
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1015-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Dec  6 18:40:13 UTC 2025

  System load:  0.0                Processes:             128
  Usage of /:   14.9% of 18.33GB   Users logged in:       1
  Memory usage: 11%                IPv4 address for enX0: 172.31.31.92
  Swap usage:   0%


Expanded Security Maintenance for Applications is not enabled.

40 updates can be applied immediately.
24 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


demo@ip-172-31-31-92:~$ exit
logout
Connection to 34.204.49.77 closed.
jenkins@ip-172-31-21-178:~$
```

Why This Step is Important

Setting up passwordless SSH is required for:

- Adding the Tomcat server as a **Jenkins agent (worker node)**
- Allowing Jenkins to:
  - Transfer files (e.g., WAR file)
  - Execute remote deployment commands
  - Restart Tomcat services if needed

Without this, the CI/CD process cannot automatically deploy the application.

2. Add SSH Credentials in Jenkins for the Worker Node

Once passwordless SSH is configured between the Jenkins Master and the Tomcat server, the next step is to **store the private key inside Jenkins**. This credential will later be used when configuring the Tomcat instance as a Jenkins agent.

This ensures secure authentication without exposing passwords and allows Jenkins to remotely execute commands on the worker node.

a) View the Jenkins User's Private Key

On the Jenkins master, display the private key: **cat ~/.ssh/id_ed25519**

You will see output similar to:

```
jenkins@ip-172-31-21-178:~$ cat .ssh/id_ed25519
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACAhOVuawvza6snTujnii5cgWqDhL9klibLKDSQkWeKkUQAAAKCpEGkqqRBp
KgAAAAtzc2gtZWQyNTUxOQAAACAhOVuawvza6snTujnii5cgWqDhL9klibLKDSQkWeKkUQ
AAAECltWOKOVUW61Hq/czD22+RAuejNZ5pdqIR2MIp+DuvfyE5W5rC/NrqydO6OeKLlyBa
oOEv2SWJssoNJCRZ4qRRAAAAGGplbmtpbnNAaXAtMTcyLTMxLTIxLTE3OAECAwQF
-----END OPENSSH PRIVATE KEY-----
jenkins@ip-172-31-21-178:~$
```

This is the key you must paste into Jenkins Credentials.

b) Open Jenkins Credentials Manager

Navigate in Jenkins:

Manage Jenkins → Credentials → System → Global credentials (unrestricted) → Add Credentials

c) Create a New SSH Credential

Fill the form as follows:

- **Kind**: SSH Username with private key
- **Scope:** Global (so any job or agent can use it)
- **ID:** Give a recognizable ID, for example: new_ssh_cred
- **Username:** Enter the SSH username of the Tomcat server, e.g.: demo

- **Private Key:** Enter directly

Then paste the entire private key from:

/var/lib/jenkins/.ssh/id_ed25519

It should look exactly like:

```
-----BEGIN OPENSSH PRIVATE KEY-----
...
-----END OPENSSH PRIVATE KEY-----
```

Leave **Passphrase empty**, because you generated the key without one.

- **Click Create:** This stores the private key securely in Jenkins and makes it available for use when a new agent node is configured.

3. Configure Worker Node in Jenkins

After setting up passwordless SSH and creating SSH credentials in Jenkins, the next step is to **add the Tomcat server as a Jenkins agent (worker node)**. This enables Jenkins to offload build or deployment tasks to the remote machine.

a) Navigate to Jenkins Node Configuration

- Go to: **Manage Jenkins → Nodes → New Node**
- Click **New Node**, then provide:
  - **Node Name:** worker_node
  - Select **Permanent Agent**
- Click **Create**.

b) Configure Node Details

Fill in the node configuration fields:

- **Name:** worker_node
- **Number of Executors:** 1 (One Job will run at a time on this node.)
- **Remote Root Directory:** This is the workspace directory on the worker node: /home/demo
- **Labels:** Labels allow Jenkins to target this specific node: slave-1 (These labels will be used later during job execution.)

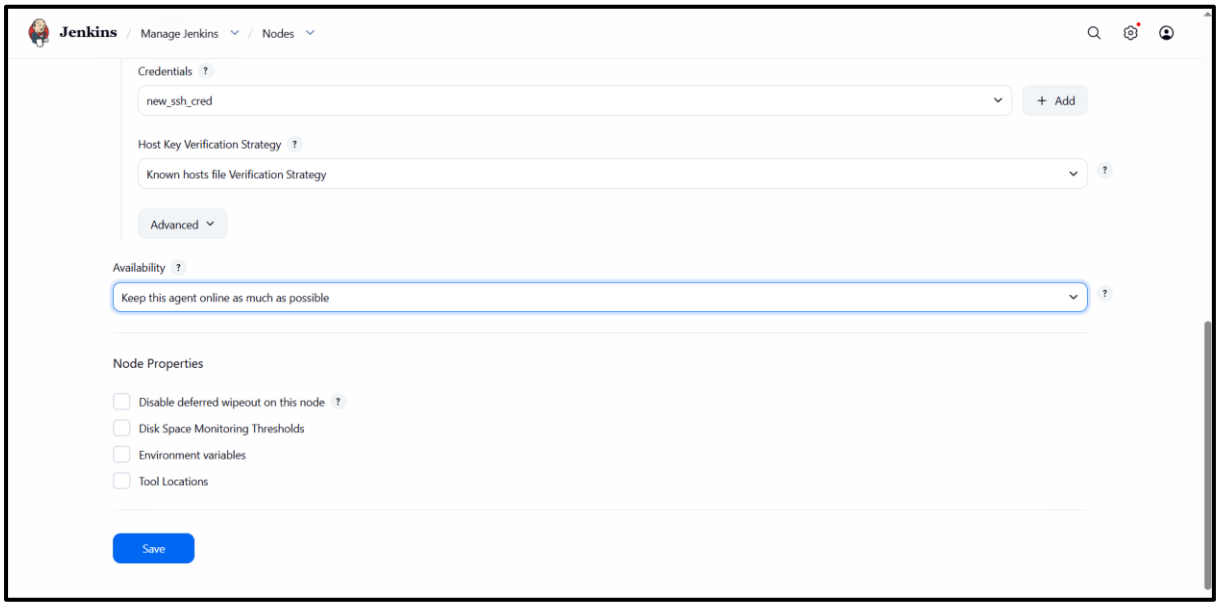
c) Configure Node Launch Method

- **Set the launch method to:** Launch agent via SSH
- **Host:** The public IP of your Tomcat EC2 instance: 34.204.49.77
- **Credentials**: Select the SSH key credential you created earlier: new_ssh_cred
- **Host Key Verification Strategy**: Choose: Known hosts file Verification Strategy (This avoids unknown host blocking issues.)

d) Availability Setting

- **Choose:** Keep this agent online as much as possible

This ensures Jenkins keeps the worker connected continuously.



e) Save the Configuration

- Click **Save**.

Jenkins now attempts an SSH connection to the Tomcat server using:

- Username: demo
- Private key stored in credentials
- Host IP: 34.204.49.77

f) Verify Node is Successfully Connected

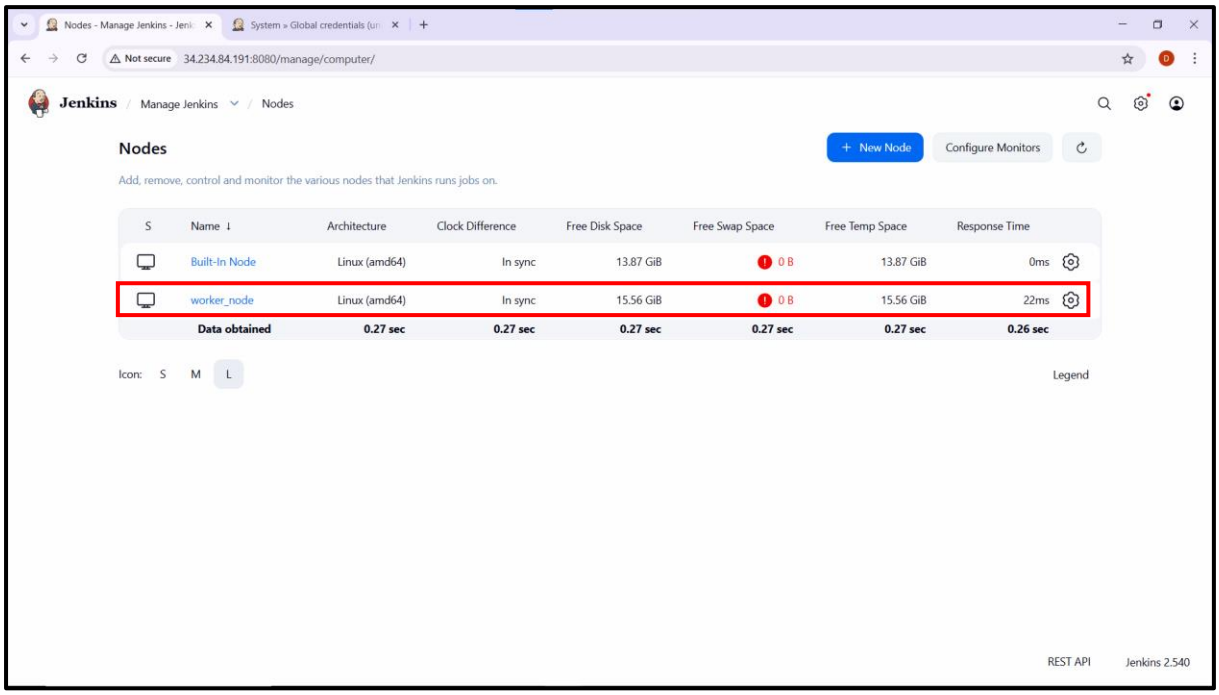
Final screenshot confirms the node was added and is online:

- Architecture detected
- Disk space info retrieved
- Response time visible

The node **worker_node** is now part of the Jenkins node list alongside:

- Built-in Node
- worker_node

This means SSH authentication succeeded and the agent is ready to execute build and deployment tasks.



Why This Step is Needed

Jenkins agents (worker nodes) require authentication so the Master can:

- Connect to the agent via SSH
- Execute tasks remotely
- Deploy artifacts (like WAR files)
- Manage workspace and job operations

By storing the private key as a Jenkins credential:

- You avoid exposing the key in jobs
- Authentication is automated and secure
- Jenkins can connect to the Tomcat server without manual intervention

Step 2: Create Jenkins Job to Build and Deploy Java Application on Tomcat

After configuring the Tomcat EC2 instance as a Jenkins worker node, the next step is to create a Jenkins job that automatically:

1. Pulls the Java application code from GitHub

2. Builds the project (creates the WAR file)
3. Deploys the WAR file to the remote Tomcat server
4. Automates the process using SCM polling

a) Install Required Jenkins Plugin

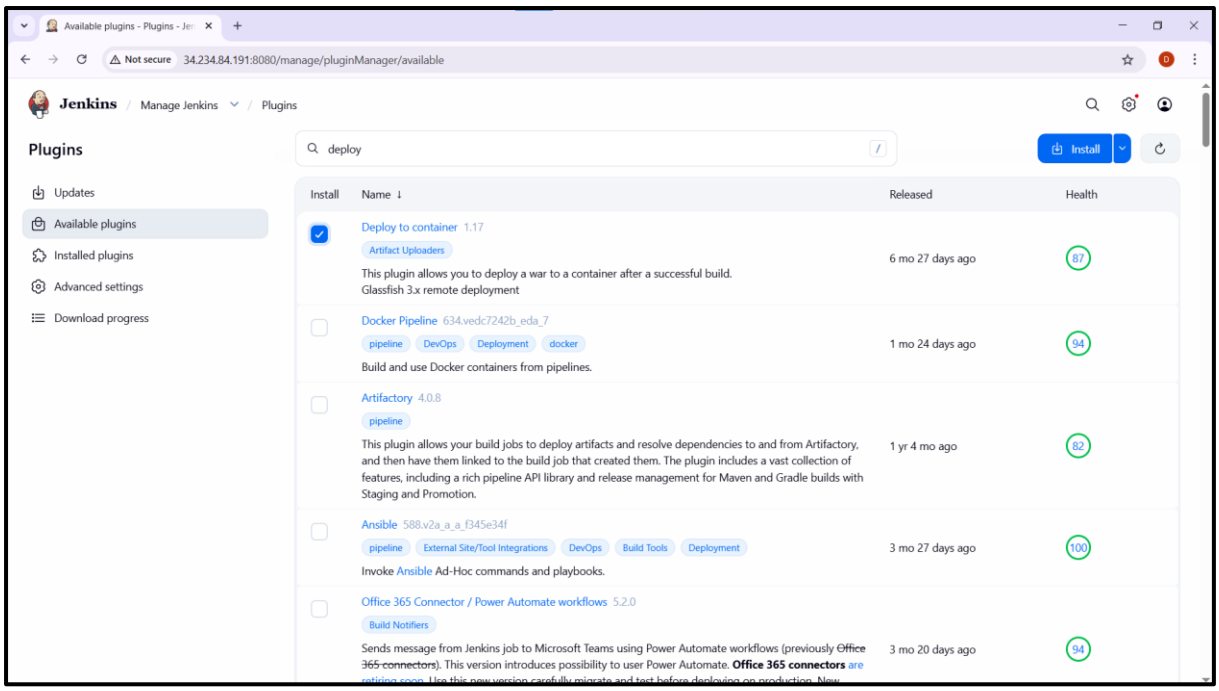
To deploy the WAR file directly to Tomcat, install: Deploy to Container Plugin

Navigate: Manage Jenkins → Plugins → Available Plugins

Search for **deploy** and install:

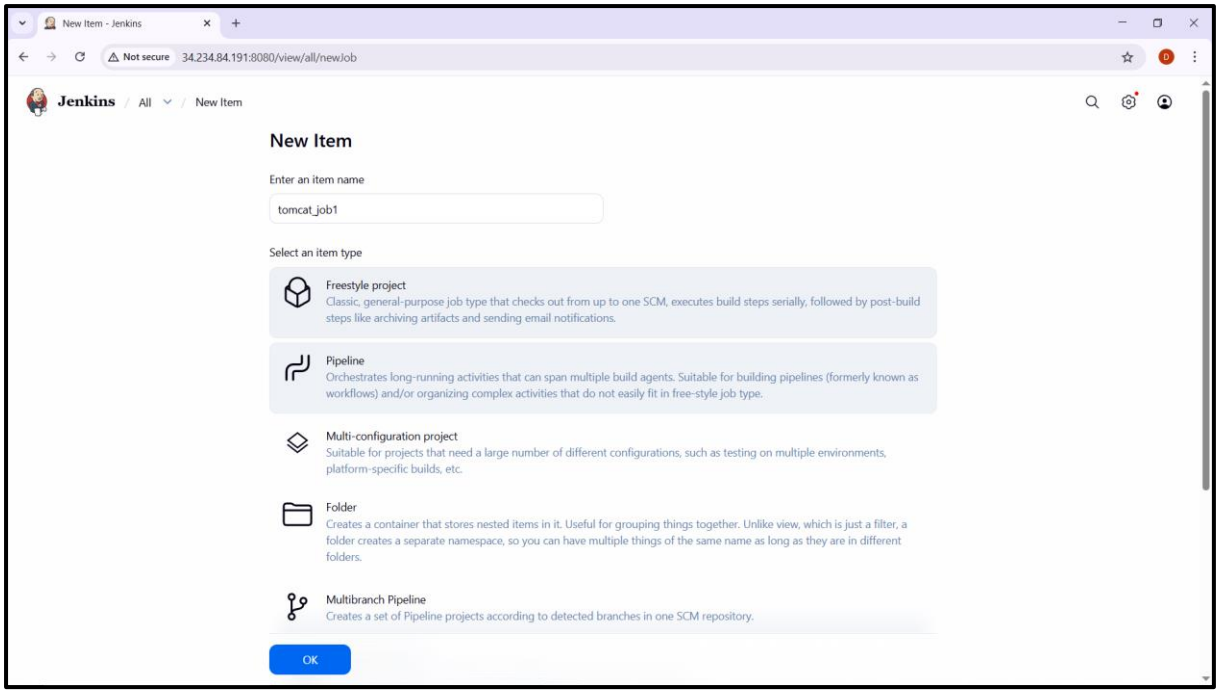
- **Deploy to container**

This plugin enables Jenkins to upload a WAR/EAR file to Tomcat Manager.



b) Create a New Jenkins Freestyle Job

- Navigate: **Jenkins Dashboard → New Item**
- Enter job name: tomcat_job1
- Select: Freestyle Project
- Click: OK



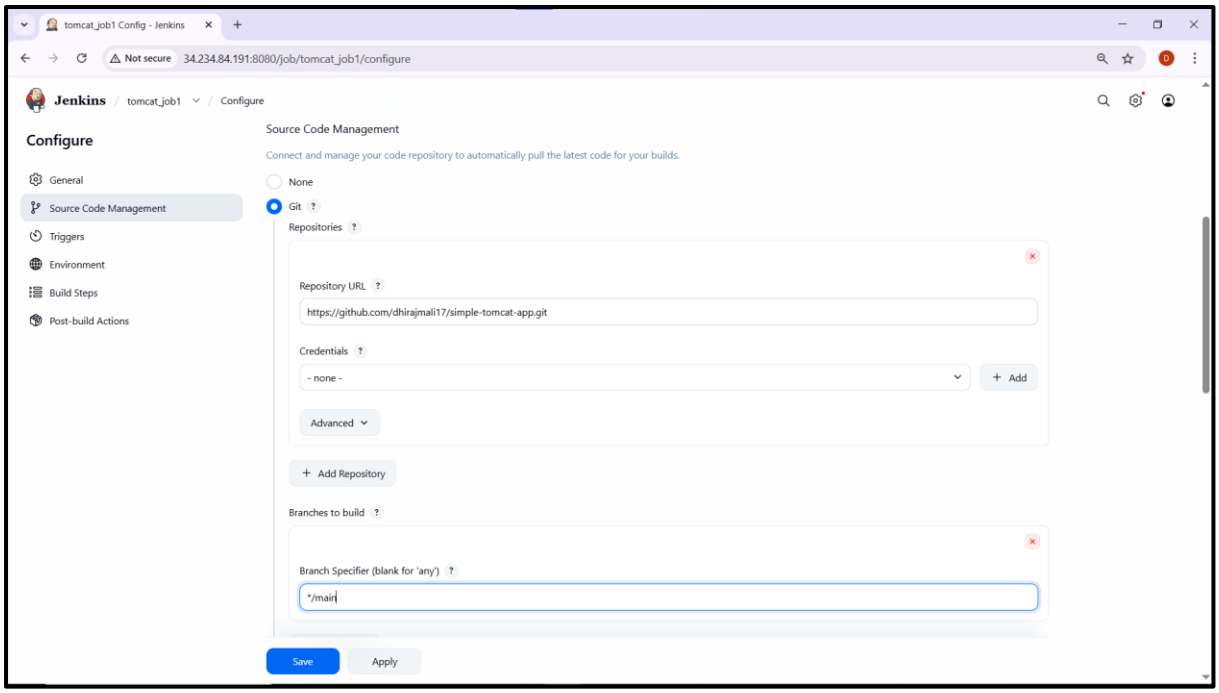
c) Configure Source Code Management (GitHub Repository)

- Under Source Code Management, select: **Git**

  Enter Repository URL: https://github.com/dhirajmali17/simple-tomcat-app.git

- Branches to build**: */main**

This ensures Jenkins always pulls the latest code from the main branch.

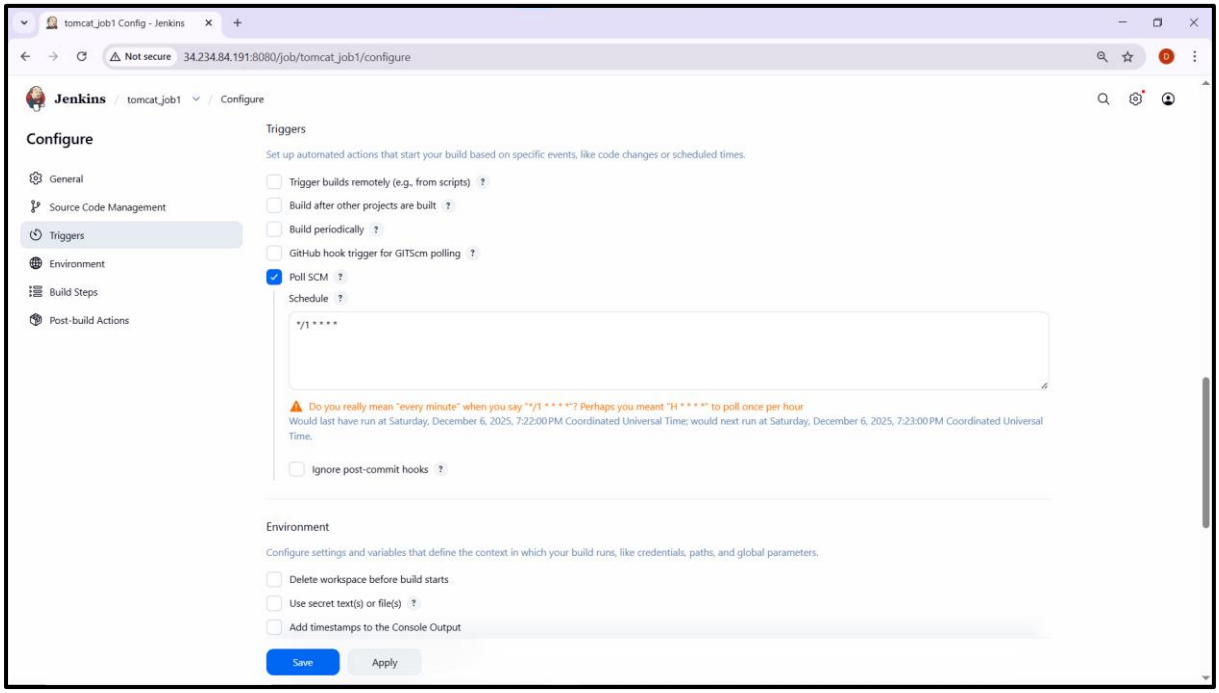


d) Configure Build Trigger (SCM Polling)

- To automate builds whenever code changes, enable: **Poll SCM**

  Enter schedule: ***/1 * * * ***

Meaning: Jenkins checks GitHub **every 1 minute** for changes.



e) Create Tomcat Deployment Credentials

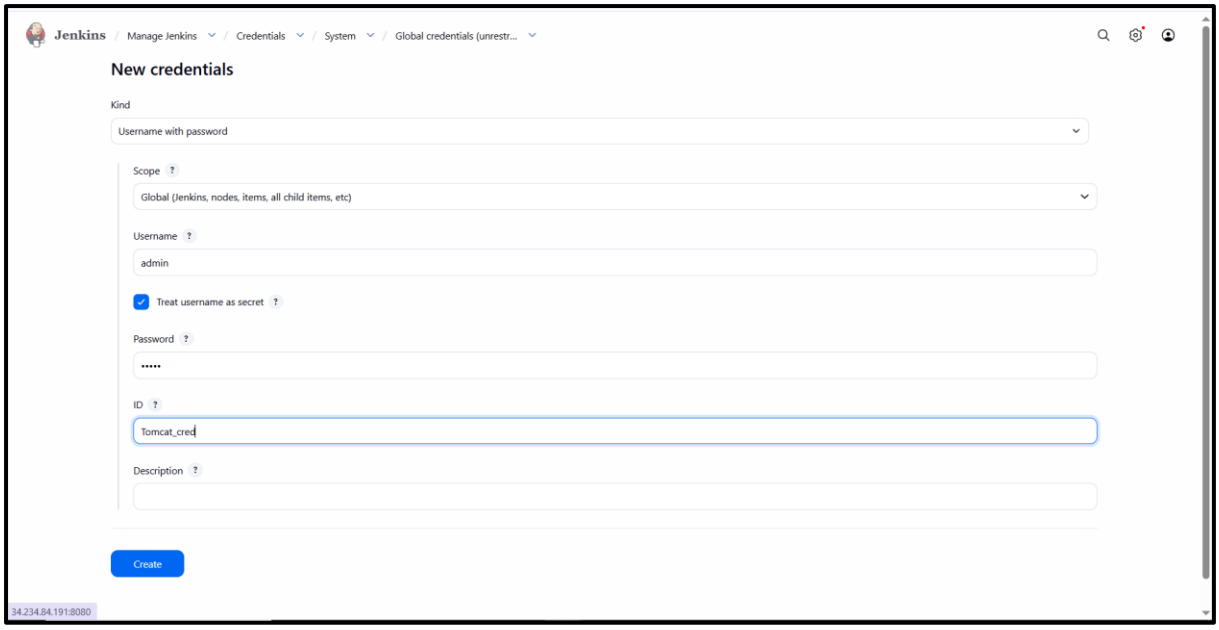
Since Tomcat Manager requires authentication, create a new credential.

Navigate: **Manage Jenkins → Credentials → Global → Add Credentials**

Select:

- Kind: **Username with password**
- Enter:
  - **Username:** admin
  - **Password:** <tomcat-admin-password>
  - **ID:** Tomcat_cred

This will be used for Tomcat Manager API deployment.



f) Configure Post-Build Deployment to Tomcat

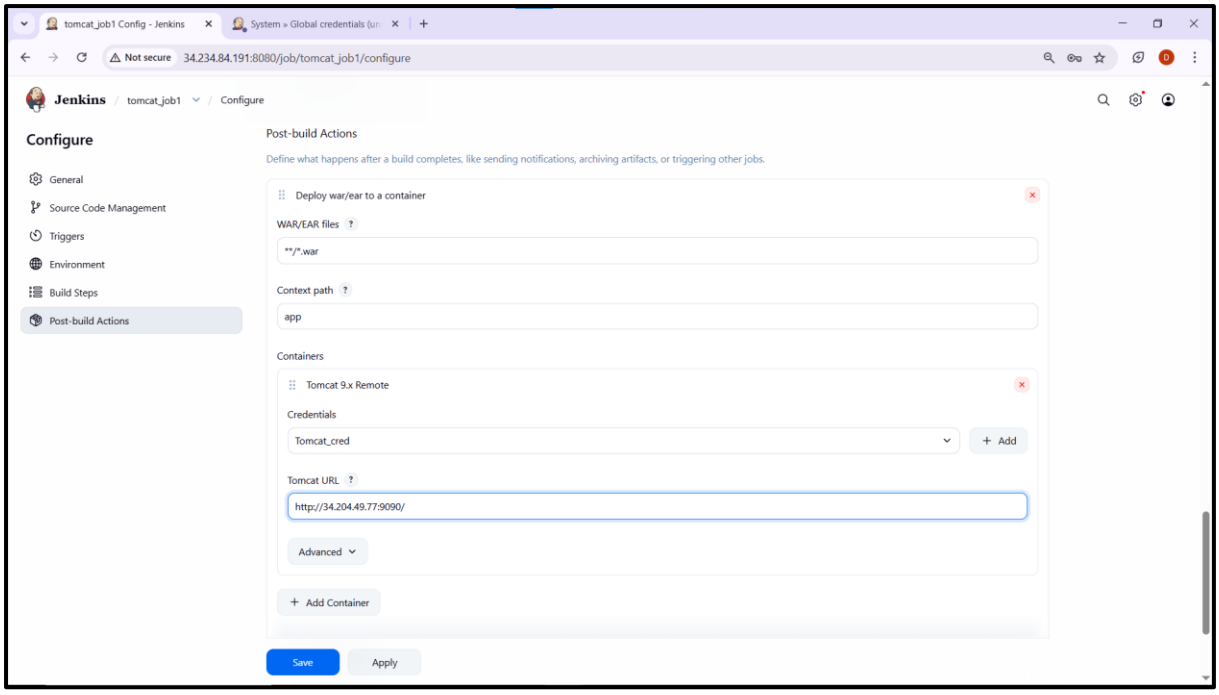
- **Go to Post-build Actions** → Click: Deploy WAR/EAR to a container
- **Configure:**
  - WAR/EAR files: **/*.war
  - Context Path: app
  - The application will be accessible at: **http://<Tomcat-IP>:8080/app/**

- **Container Type:**
  - Select: Tomcat 9.x Remote

- **Then configure:**

  - Credentials: Select **Tomcat_cred**
  - Tomcat URL: **http://34.204.49.77:9090/**

Ensure Tomcat Manager is running on port 9090.



g) Save Configuration and Run the Job

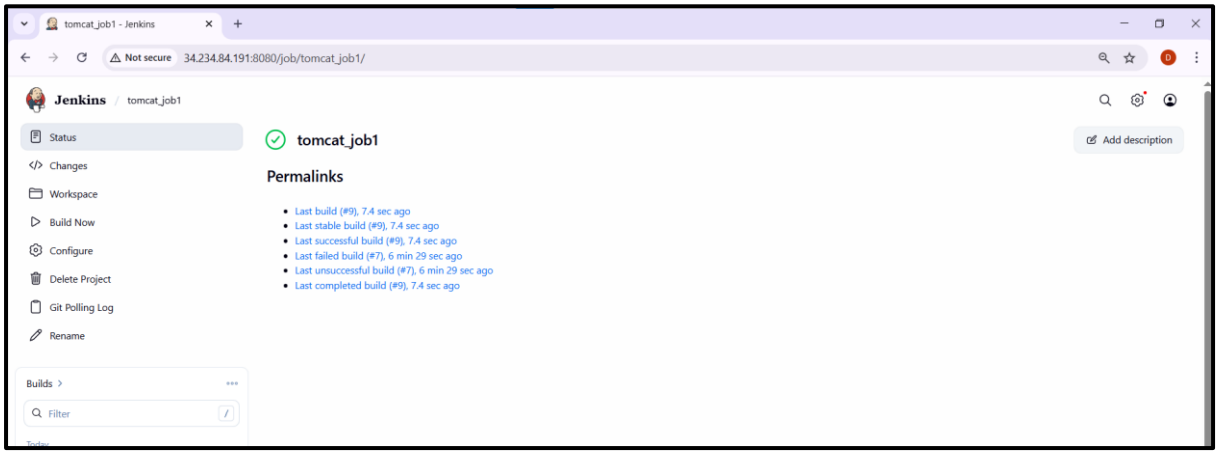
Click **Save** → **Build Now**

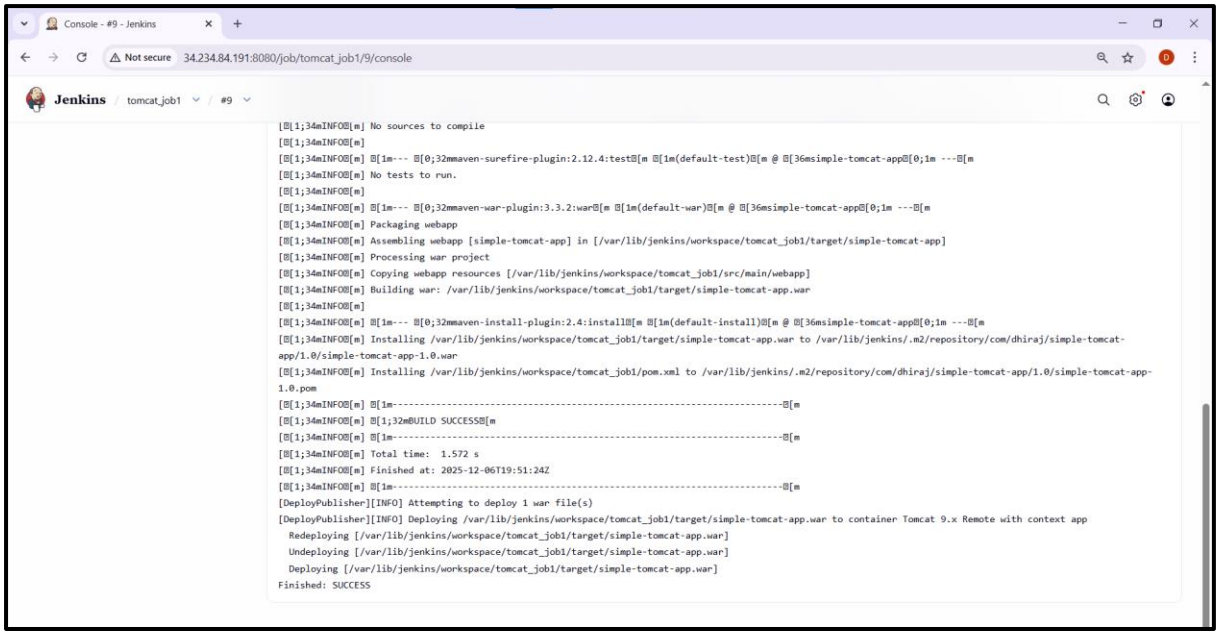
Jenkins will:

1. Pull code from GitHub
2. Build the WAR file
3. Connect to Tomcat via Tomcat Manager
4. Deploy the application automatically

final screenshot confirms:

- Build successful
- Deployment successful
- Job running on Jenkins worker node





Step 3: Verify Deployment on Tomcat

After the Jenkins job successfully deployed the WAR file, the final step is to verify the application on the Tomcat server.

a) Check Application in Tomcat Manager

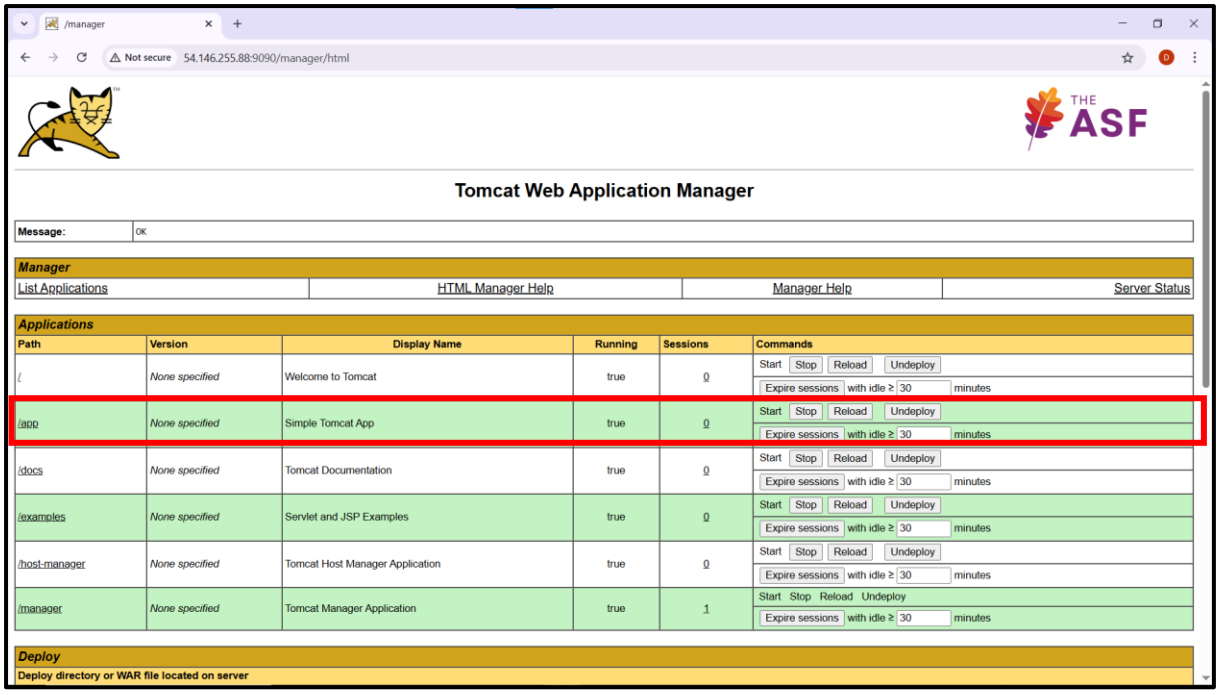
Login to: **http://<Tomcat-IP>:9090/manager/html**

- Using the credentials added in Jenkins (admin / admin).
- Under the **Applications** list, the deployed app appears as:

  /app → Simple Tomcat App

  Status: Running

This confirms the WAR file was deployed correctly by Jenkins.



b) Access the Application in Browser

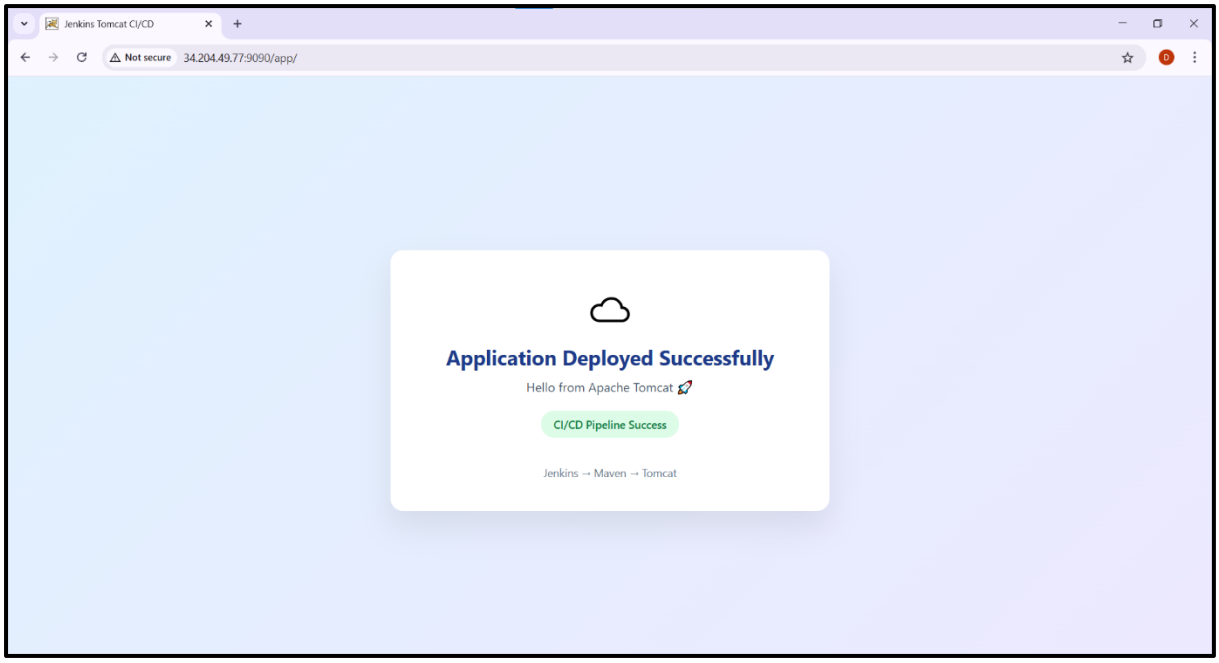
Open the app URL: http://<Tomcat-IP>:9090/app/

You should see:

"Application Deployed Successfully"

This confirms the entire CI/CD workflow is working:

- Jenkins pulled code
- Maven built the WAR
- Jenkins deployed to Tomcat
- Application is live



- **Conclusion**

This project successfully implemented a multi-node automated build and deployment system using Jenkins, Maven, and Apache Tomcat. Jenkins pulled the Java application from GitHub, built it into a WAR file, and deployed it automatically to the Tomcat server. The deployment completed without issues, and the application became accessible in the browser, confirming that the automated workflow worked as expected. Overall, this setup demonstrates essential DevOps practices such as automation, continuous integration, continuous deployment, secure SSH communication, and distributed node configuration. It provides a strong foundation for scaling into more advanced CI/CD pipelines, container-based deployments, and cloud-native environments.